MASTER BACKEND SPECIFICATION

Real-Estate Consultant System

Production System Specification & Engineering Standards — Version 1.0

Target: Single-Operator Consultancy **Stack:** FastAPI / Pydantic v2 / PostgreSQL **Status:** Active Source of Truth

1. PROJECT OBJECTIVE

Build a professional web-based system for a one-person real-estate brokerage and consultancy business. The business owner currently handles the complete end-to-end operation:

- Property brokerage
- Property buying / selling assistance
- Rental assistance
- Property sourcing
- Property listing and marketing
- Customer enquiries
- Property requirement matching
- Site-visit coordination
- Property-document assistance
- Preliminary document review
- Registration & process coordination
- Lead follow-ups
- Deal management
- Brokerage & service-fee tracking

Core Deliverables:

1. **Public-facing website:** High-trust discovery portal for customers.
2. **Private authenticated dashboard:** Comprehensive workspace for the business owner.
3. **Reliable backend API:** Robust, secure REST endpoints on `/api/v1`.
4. **Secure private document management:** Ephemeral signed access and strict validation.
5. **Auditability & fault tolerance:** Immutable business audit trails and safe failure semantics.
6. **Modular architecture:** Adaptable for future organizational growth without rewrites.

2. PRODUCT PHILOSOPHY

This is **NOT** a generic real-estate marketplace, nor is it intended to compete with large property portals.

> **Personal Real-Estate Consultant Website + Business Management System**

The business owner remains the primary operator. V1 must be simple enough for one person to operate efficiently while the backend architecture must be clean enough to support future expansion.

> **Important Principle:** Build for the client's CURRENT requirements while keeping the architecture extensible. Do NOT implement speculative features merely because they might be useful someday.

3. V1 USER TYPES

3.1 Public Visitor (Unauthenticated)

- Browse, search, and filter published properties
- View property details and public CDN images
- Submit general enquiries or property-specific queries

3.2 Business Owner / Consultant (Authenticated)

- Full property inventory & image management
- Client & lead lifecycle tracking
- Property requirement recording & rule-based matching

- Request specific properties or site visits
- Request professional consultation
- Access contact channels (WhatsApp / direct call links)
- Site visit & follow-up scheduling
- Document vault & preliminary reviews
- Deal execution & commission/service-fee tracking
- Operational dashboard metrics & audit history review

3.3 Future Users

The system must support future actors (Staff, Additional Consultants, Assistants, Branch Managers). While V1 does not require complex multi-user workflows, authentication and authorization must never be hard-coded around a single-user assumption.

4. TECHNOLOGY ARCHITECTURE

Component	Specification / Standards
Backend Runtime	Python 3.11+, FastAPI, Pydantic v2
Database & ORM	PostgreSQL (15+), SQLAlchemy 2.x (Async/declarative), Alembic migrations
Authentication	Argon2id password hashing, Short-lived JWT access tokens, Rotating refresh tokens with revocation support
Storage Engine	S3-compatible Object Storage (never store binary files as DB blobs)
API Protocol	RESTful JSON, Base path: `/api/v1`
Layering Model	`Router → Schema Validation → Service → Repository → Database` (Zero business logic in route handlers)

5. BACKEND MODULE STRUCTURE

The backend enforces clear single-responsibility domain modules:

- **Auth:** Session, token rotation, authentication
- **Users:** User profiles, credentials, role mapping
- **Properties:** Inventory records, listing states
- **Property Images:** Visual media metadata, order
- **Clients:** Contact records and client history
- **Leads:** Sales funnel and lead qualification
- **Property Requirements:** Customer search criteria
- **Documents:** Secure metadata and private files
- **Verification:** Preliminary document checks & logs
- **Site Visits:** Visit scheduling and conflict guards
- **Follow-ups:** Task management and reminders
- **Deals:** Closed transaction and financial tracking
- **Commissions / Fees:** Segregated revenue accounting
- **Notifications:** Alerts, email, and messaging hooks
- **Audit Logs:** Immutable business event journal
- **System Settings:** System-wide dynamic configurations

6. PROPERTY MANAGEMENT

Represents real estate inventory marketed, evaluated, or managed by the consultant.

> **Entity Attributes:** Internal Property UUID, Public Reference ID (e.g., `PR-2026-000123`), Title, Property Type, Transaction Type, Description, Price, Price Negotiability, Built-up/Plot Area, Area Unit (sq.ft, cents, acres), Bedrooms, Bathrooms, Floor, Total Floors, Facing (North, South, East, West, etc.), Furnishing State, Parking Spaces, Property Age, District, City, Taluk, Locality, Pincode, Coordinates (Lat/Long), Google Maps Reference, Inventory Source, Owner Information (name, contact, authorization), Advertisement Authorization, Internal Notes, Last Verified Date, Timestamps.

> **Security Mandate:** Sensitive owner personal information and internal evaluation notes must NEVER be exposed through public APIs.

7. PROPERTY TYPES

Configurable domain data (not hard-coded enums):

- Apartment
- Independent House / Villa
- Residential Plot / Layout Land
- Commercial Property / Office Space / Shop
- Agricultural Land / Farmhouse
- Other / Mixed Use

8. TRANSACTION TYPES

- `SALE` Outright sale / acquisition
- `RENT` Residential / commercial rental
- `LEASE` Long-term commercial or land lease

9. PROPERTY LIFECYCLE

Lifecycle Flow:

`DRAFT → PUBLISHED → PAUSED → SOLD / RENTED → ARCHIVED`

- **DRAFT:** Internal preparation; hidden publicly.
- **PUBLISHED:** Live and searchable on public portal.
- **PAUSED:** Temporarily unlisted from search.
- **SOLD** / **RENTED:** Unavailable for new enquiries.
- **ARCHIVED:** Soft-deleted historical record.

> Never physically hard-delete historical inventory. Use archival flags.

10. PROPERTY IMAGES

Separate from legal/business documents. Images are hosted on public storage/CDN distribution.

- **Metadata:** Image ID, Property ID, Storage Key, Original Filename, MIME Type, File Size, Checksum, Display Order, Primary Image Flag, Upload Timestamp, Uploaded By.
- **Capabilities:** Multi-image upload, reordering, primary selection, deletion/archival.

11. CLIENT MANAGEMENT

- **Profile:** Client UUID, Full Name, Phone, Email, Preferred Contact Method, Postal Address, Client Classification (Buyer, Seller, Tenant, Landlord, Investor), Source, Status.
- **Privacy:** Strict separation from public serialization. Collect only essential contact details.

12. LEAD MANAGEMENT

Sources: Website, Direct Call, WhatsApp, Referral, Direct Walk-in.

Funnel:
`NEW → CONTACTED → INTERESTED → SITE_VISIT → NEGOTIATION → CONVERTED`
Terminal fallback: `LOST` (with captured lost reason).

13. PROPERTY REQUIREMENTS

Stored as independent records linked to clients to allow historical matching:

- Transaction type (Buy / Rent / Lease)
- Target locations, districts, and localities
- Property categories, budget bounds (min/max)
- Area preferences, bedroom/bathroom counts
- Facing orientation, furnishing requirements

14. PROPERTY MATCHING

Rule-based algorithmic matching in V1 (filtering properties against stored requirement parameters). AI matching is deliberately deferred to avoid premature complexity.

15. ENQUIRIES

- Public submission (no login required).
- Captures visitor name, contact, message, property ref, source.
- Automatically creates or attaches to a lead entity.
- Protected by IP rate-limiting, honeypots, and size restrictions.

17. FOLLOW-UPS

Task tracking system for operational momentum:

- Target: Lead or Client
- Related Property (optional)
- Scheduled execution timestamp

16. SITE VISITS

Model: Customers *request* visits; system must not auto-confirm.

`REQUESTED → CONFIRMED → COMPLETED`
Exceptions: `CANCELLED` | `RESCHEDULED`

Concurrency guards prevent conflicting double-bookings on the same slot.

- Action type: Call, Send Docs, Arrange Visit, Owner Follow-up, Missing Paperwork
- Status: Scheduled, Completed, Missed
- Completion notes and timestamp

18–22. SECURE DOCUMENT MANAGEMENT ARCHITECTURE

Storage & Access Paradigm: Binary documents (Sale Deeds, Parent Documents, Encumbrance Certificates, Patta/Chitta, Tax Receipts, Approved Plans, Identity Proofs) are strictly sequestered in private object storage. PostgreSQL maintains metadata only.

```
Private S3 Storage + PostgreSQL Metadata  |  Zero Public File URLs  |  Short-Lived Signed URLs Only
```

Dimension	Enforced Security & Validation Standard
Upload Validation	1. Extension whitelist (e.g., `.pdf`, `.jpg`, `.png`) 2. Magic byte / MIME inspection (never trust client header) 3. Size limits & zero-byte detection 4. SHA-256 checksum calculation (duplicate identification) 5. UUID-based storage key generation (prevents path traversal)
Access Control	Authenticated endpoint → RBAC permission check → Ephemeral pre-signed S3 URL (max 15 mins). No permanent paths.
Document Lifecycle	`UPLOADED → UNDER_REVIEW → PRELIMINARILY_CHECKED` (or `REQUIRES_ATTENTION`). Physical deletes prohibited; soft-archive only.

23. PRELIMINARY DOCUMENT REVIEW

System aids consultant in organizing verification workflows:

- Standardized preliminary checklist
- Checklist items linked to specific property docs
- Consultant observations and missing document notes
- Status: `PRELIMINARILY_CHECKED`, `REQUIRES_ATTENTION`, `INCOMPLETE`

24. LEGAL-SAFE PRODUCT LANGUAGE

Strict Compliance Guardrail:
Never use: *"Government Approved"*, *"100% Legally Verified"*, *"Guaranteed Clear Title"*, or *"Risk-Free Property"*.

Permitted terminology: *"Document Assistance"*, *"Preliminary Document Review"*, *"Property Document Coordination"*. System must explicitly advise clients to seek qualified legal counsel.

25–26. DEALS & COMMISSION ACCOUNTING

Deal Entity

- Linked Client, Property, Deal Type
- Agreed Transaction Amount (Decimal)
- Execution Date, Deal Status
- Brokerage Fee & Consultant Service Fee

Financial Data Integrity Rules

- **Mandatory:** Use `NUMERIC / Decimal`. Floating point math is strictly forbidden.
- Segregate official government charges (stamp duty, registration fees) from consultant fees.

```
Transaction Total = Government/Official Charges + Consultant Service Fee + Brokerage Commission
```

27–28. AUTHENTICATION & AUTHORIZATION (RBAC)

Authentication Mechanics

- Argon2id password hashing (RFC 9106)
- Stateless JWT access tokens (15–30 min life)
- Opaque refresh tokens stored in DB with hash
- Refresh token rotation upon every renewal
- Immediate token revocation on logout / anomaly
- Centralized rate limiting on auth routes

Internal Granular Permissions

Hard-coded single-user shortcuts are prohibited. RBAC permits clean multi-user expansion:

```
PROPERTY_READ, PROPERTY_WRITE, PROPERTY_PUBLISH,
CLIENT_READ, CLIENT_WRITE, LEAD_READ, LEAD_WRITE,
DOCUMENT_READ, DOCUMENT_WRITE, VERIFICATION_WRITE,
SITE_VISIT_WRITE, DEAL_WRITE, AUDIT_READ,
SETTINGS_WRITE
```

29–30. PUBLIC VS PRIVATE BOUNDARIES & API RESPONSE CONTRACT

Strict schema isolation ensures internal columns, owner contact info, and internal notes are never serialized to public consumers.

Standard Success Envelope (HTTP 200/201):

```
{
  "success": true,
  "data": { ... },
  "message": "Operation completed successfully."
}
```

Standard Error Envelope (HTTP 4xx/5xx):

```
{
  "success": false,
  "error": {
    "code": "PROPERTY_NOT_FOUND",
    "message": "Property not found."
  }
}
```

Standardized machine-readable error codes: `UNAUTHORIZED`, `FORBIDDEN`, `VALIDATION_ERROR`, `RESOURCE_CONFLICT`, `RATE_LIMITED`, `INTERNAL_ERROR`. Never leak stack traces, SQL syntax, or server paths.

31–36. DATABASE, TRANSACTIONS & CONSISTENCY RULES

- **Identifier Strategy:** Secure UUIDs for internal primary keys. Human-readable public references (e.g. `PR-2026-000123`) for customer-facing interfaces. Sequential DB IDs must never be exposed.
- **Multi-Record Atomicity:** Operations affecting multiple records (e.g., Deal + Commission + Audit) must run inside strict SQLAlchemy transactional context managers. Partial writes are prohibited.
- **Storage + Database Consistency:**
  - *Storage OK, DB Fails:* Compensating transaction automatically purges the orphaned object from S3.
  - *DB Created, Storage Fails:* Prohibited pattern. Files are written and validated before finalizing metadata.
  - *Interrupted Uploads:* Partial objects are cleaned; no phantom DB rows created.

- **Soft Delete & Immutability:** Business records are archived using state indicators or soft-delete timestamps. Audit logs are append-only and strictly immutable.

37–39. FAULT TOLERANCE, RETRIES & CONCURRENCY

Fault Tolerance & Safe Retries

- Zero blanket `except Exception: pass`.
- Centralized exception handler maps domain exceptions to standard HTTP error envelopes.
- Automatic retries restricted to transient infrastructure operations (network glitch, S3 timeout).
- Idempotency keys enforced on payment, lead submission, and site-visit creation.

Concurrency & Race Protection

- PostgreSQL row-level locking (`SELECT ... FOR UPDATE`) on critical state transitions.
- Database unique constraints prevent double bookings and duplicate submissions at the DB engine level.
- Optimistic concurrency checks where locking is inappropriate.

40–42. AUDIT LOGGING, DIAGNOSTICS & HEALTH CHECKS

Audit Log Schema

Captures actor ID, action name, entity type, entity UUID, change diff, request IP, and correlation ID.

```
PROPERTY_CREATED, PROPERTY_PUBLISHED,
DOCUMENT_UPLOADED, LEAD_CONVERTED,
SITE_VISIT_CONFIRMED, DEAL_CLOSED
```

Secrets, tokens, and PII must be sanitized before recording.

Operational Health Probes

- `GET /api/v1/health/live`: Process heartbeat. Returns 200 if ASGI server is running.
- `GET /api/v1/health/ready`: Dependency verification. Validates active DB pool connection and S3 connectivity before accepting traffic.

43–46. CONFIGURATION, SECURITY & PRIVACY BASELINES

- **Configuration (12-Factor):** All settings loaded via Pydantic `BaseSettings` from environment variables. Strict validation at boot. Real credentials banned from source control.
- **Strict CORS:** Wildcard (`*`) is banned on authenticated routes. Explicit whitelist configured per environment (Local Dev, Staging, Production).
- **Defense-in-Depth:** Rate-limiting on public routes, Argon2id, secure HTTP headers, ORM parameterization (no raw SQL concat), file magic-number validation, maximum payload size enforcement.

47. NOTIFICATIONS

- Lightweight transactional notifications (Email via SMTP/SES, Dashboard alerts, WhatsApp deep links).
- Decoupled interface allows future plug-and-play SMS/WhatsApp automation without touching domain logic.

48. OPERATIONAL DASHBOARD

- Aggregate stats: Active inventory, pending leads, confirmed visits, reviews in progress, open deals.
- Direct SQL aggregation (COUNT, SUM) — no full-table in-memory loading.

49. SEARCH & PAGINATION

- Indexed compound queries on Location, Property Type, Budget, Bedrooms, and Listing Status.
- Offset/limit and cursor pagination supported across all collection endpoints. Unbounded queries are blocked.

Recommended Base Route Tree:

`/api/v1/auth` | `/api/v1/properties`
`/api/v1/leads` | `/api/v1/documents`
`/api/v1/site-visits` | `/api/v1/deals`
`/api/v1/verifications` | `/api/v1/audit`

50–54. COMPREHENSIVE TESTING STRATEGY

Mandatory Testing Matrix (CI/CD Quality Gate):

Test Layer	Target Behaviors & Validation Requirements
Unit Tests	Domain validators, Pydantic schemas, financial fee calculations, permission matrix checks, state machine transitions.
Integration Tests	Full API roundtrips, PostgreSQL transaction rollbacks, authenticated session flows, presigned URL generation.
Property Suite	Valid creation, invalid price/area validation, status progression, anonymous public vs auth owner response filtering.
Leads & Visits	Duplicate lead prevention, conflicting site-visit booking rejections, reschedule state transitions, rate limit throttling.
Document Suite	MIME spoofing rejection, oversized file blocking, SHA-256 duplicate detection, storage failure rollback & cleanup.
Failure & Fault Tests	DB pool exhaustion, S3 network timeout handling, concurrent updates, optimistic lock collision recovery.

55. DEPLOYMENT REQUIREMENTS

- Dockerized container running behind reverse proxy (Nginx / Caddy) with TLS 1.3.
- Automated PostgreSQL daily snapshots with periodic restore rehearsals.
- Alembic migrations executed during zero-downtime deployment pipelines.
- Non-root container user execution.

56. PROJECT STRUCTURE

```
backend/
├── app/
│   ├── api/ (deps.py, v1/...)
│   ├── core/ (config, security, log)
│   ├── db/ (session, base)
│   ├── models/ (SQLAlchemy)
│   ├── schemas/ (Pydantic v2)
│   ├── repositories/
│   ├── services/
│   └── main.py
├── migrations/ (Alembic)
├── tests/ (unit, integration, failure)
├── docker-compose.yml
└── Dockerfile
```

57–59. EVOLUTION STRATEGY & SCOPE BOUNDARIES

Architectural Future-Proofing (Supported)

- Multiple staff & agent RBAC assignments
- Customer self-service portal (read-only requirements)
- Owner self-submission portal
- Multi-branch & regional expansion
- AI matching service over existing schema
- Integrated payment gateway / invoicing

Strictly Out of Scope for V1

- Microservices or Kubernetes clusters
- Automated legal title clearance
- Scraping government revenue portals
- WhatsApp Business API automated bots
- Online payment gateways & escrow accounts
- Event-driven Kafka / RabbitMQ pipelines

60–64. IMPLEMENTATION GOVERNANCE & DEFINITION OF DONE

Implementation Order:

1. SPEC → 2. ARCH → 3. FOUNDATION → 4. DB/ALEMBIC → 5. AUTH → 6. PROPERTIES → 7. LEADS → 8. REQUIREMENTS → 9. DOCS/STORAGE → 10. VERIFICATION → 11. SITE VISITS → 12. DEALS → 13. AUDIT → 14. TESTING → 15. FREEZE

Implementation Rules

- Never redesign architecture mid-implementation.
- Master specification is the absolute source of truth.
- Never bypass repository/service layer boundaries.
- Never use floating point numbers for financial data.
- Never commit credentials or tokens to git.
- Document all backward-compatible changes clearly.

Definition of Done (Per Module)

- SQLAlchemy models + Alembic migrations checked in.
- Pydantic v2 schemas defined (input, output, internal).
- Service + Repository layers implemented cleanly.
- Granular RBAC authorization hooks active.
- Audit events published on mutations.
- Unit, integration, and failure test suites passing.

```
Simple V1 + Strong Fundamentals + Clear Module Boundaries + Secure Data Handling = Change-Resistant System
```